

AI Lab Test - 2

Solved Submission

Date: 29/09/26

1. CSP: Laboratory Scheduling

The problem has three section variables A, B and C. Each variable represents a complete assignment (time slot, lab). The domains below already include the unary restrictions from the question.

Variable	Domain
A	(1,L1), (2,L1), (3,L1)
B	(2,L1), (2,L2), (3,L1), (4,L2)
C	(1,L2), (4,L2)

Unary constraints

A uses only time slots 1, 2, 3 and must use L1. B uses only 2, 3, 4; L2 is unavailable in slot 3, and if B is in slot 4 it must use L2. C uses only 1, 3, 4, cannot use L1, and therefore its possible assignments after the lab/time restrictions are (1,L2) and (4,L2).

Binary constraints

- A and B cannot have the same time slot.
- B and C cannot have the same time slot.
- Two sections cannot use the same lab at the same time.
- A and C cannot be consecutive in time.

Backtracking with MRV and Forward Checking

- Initial MRV values: C = 2, A = 3, B = 4. C is selected.
- Try C = (1,L2). Forward checking removes (2,L1) from A because A and C cannot be consecutive.
- Remaining domains: A = {(1,L1),(3,L1)} and B = {(2,L1),(2,L2),(3,L1),(4,L2)}.
- MRV selects A because its domain has 2 values.
- Try A = (1,L1). No remaining domain becomes empty.
- MRV selects B.
- Try B = (2,L1). All variables are assigned and every constraint is satisfied.
- No backtracking is needed for this branch. No domain becomes empty.

Final schedule

Section	Time Slot	Lab
A	1	L1
B	2	L1
C	1	L2

2. TSP using Genetic Algorithm

A chromosome is a permutation of the 20 cities with city 1 fixed as the hub. The tour returns to city 1 after visiting all other cities exactly once. Fitness is $1 / \text{total tour distance}$.

The supplied PDF gives the X,Y coordinates of all 20 cities on page 7. The separate Excel file mentioned in the question was not supplied with the uploaded material, so the submitted program builds the Euclidean distance matrix directly from those coordinates.

Genetic algorithm used

- Initial population: random tours.
- Selection: roulette wheel using fitness = $1 / \text{distance}$.
- Crossover: Order Crossover (OX).
- Mutation: swap mutation with probability 0.15.
- Elitism: the best tour of every generation is copied directly to the next population.
- Random seed: 42 so the shown results are reproducible with the submitted program.

Execution results

Case	Population	Generations	Best tour	Distance
A	10	100	1-3-16-12-11-19-6-4-14-15-8-2-5-17-10-18-9-20-13-7-1	804.00
B	20	100	1-10-3-18-17-12-19-6-4-11-14-8-2-15-16-5-20-9-13-7-1	826.37
C	30	100	1-13-7-20-17-5-3-18-16-11-6-4-2-8-19-15-14-10-12-9-1	944.32
D	10	200	1-10-14-5-12-11-19-6-2-8-15-4-16-17-18-9-13-3-20-7-1	817.58
E	20	200	1-10-20-7-13-18-5-17-14-11-19-6-4-8-2-12-16-15-3-9-1	912.51
F	30	200	1-9-3-2-8-6-14-15-11-19-4-16-10-20-7-17-5-12-18-13-1	984.68

Because the genetic algorithm is stochastic, changing the random seed or mutation/crossover choices can produce different valid routes and distances. The table above corresponds to the exact submitted program.

3. Source Files

CSP.cpp contains the backtracking search with MRV, degree tie-breaking and forward checking. TSP_GA.cpp contains the complete genetic algorithm using vectors and ordinary functions without custom structs.

CSP.cpp

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> domain = {{11,21,31},{21,22,31,42},{12,42}};
vector<int> current(3,-1), used(3,0);
string name[3]={"A","B","C"};

int timeOf(int x){ return x/10; }
int labOf(int x){ return x%10; }

bool valid(int a,int b,int c){
    if(a!=-1 && b!=-1 && timeOf(a)==timeOf(b)) return false;
    if(b!=-1 && c!=-1 && timeOf(b)==timeOf(c)) return false;
    if(a!=-1 && c!=-1){
        if(timeOf(a)==timeOf(c) && labOf(a)==labOf(c)) return false;
        if(abs(timeOf(a)-timeOf(c))==1) return false;
    }
    return true;
}

bool pairValid(int v,int value,int u,int other){
    int t1=timeOf(value), t2=timeOf(other);
    if((v==0 && u==1) || (v==1 && u==0)) return t1!=t2;
    if((v==1 && u==2) || (v==2 && u==1)) return t1!=t2;
    if((v==0 && u==2) || (v==2 && u==0))
        return abs(t1-t2)!=1 && !(t1==t2 && labOf(value)==labOf(other));
    return true;
}

int degree(int v){
    int d=0;
    for(int u=0;u<3;u++) if(u!=v && !used[u]) d++;
    return d;
}

int chooseVariable(){
    int best=-1;
    for(int i=0;i<3;i++){
        if(used[i]) continue;
        if(best==-1 || domain[i].size()<domain[best].size() ||
            (domain[i].size()==domain[best].size() && degree(i)>degree(best))) best=i;
    }
    return best;
}

void printDomain(){
    for(int i=0;i<3;i++){
        if(used[i]) continue;
        cout << name[i] << ": ";
        for(int x:domain[i]) cout << "(" << timeOf(x) << ", L" << labOf(x) << ") ";
        cout << "\n";
    }
}

bool solve(){
    int left=0;
    for(int i=0;i<3;i++) if(!used[i]) left++;
    if(left==0) return true;

    int v=chooseVariable();
    cout << "MRV selects " << name[v] << "\n";

    vector<int> old=domain[v];
    for(int value:old){
        bool good=true;
        for(int u=0;u<3;u++) if(used[u]){
            if(!pairValid(v,value,u,current[u])) good=false;
        }
        if(!good){
            cout << "Try " << name[v] << " = (" << timeOf(value) << ", L" << labOf(value) << ") -> constraint violated\n";
            continue;
        }
    }
}
```

```

    cout << "Try " << name[v] << " = (" << timeOf(value) << ", L" << labOf(value) << ")\n";
    used[v]=1;
    current[v]=value;

    vector<vector<int>> oldDomains=domain;
    bool empty=false;
    for(int u=0;u<3;u++) if(!used[u]){
        vector<int> nd;
        for(int x:domain[u]) if(pairValid(v,value,u,x)) nd.push_back(x);
        domain[u]=nd;
        if(nd.empty()){
            cout << "Forward checking: domain of " << name[u] << " becomes empty\n";
            empty=true;
        }
    }

    if(!empty){
        cout << "Domains after forward checking:\n";
        printDomain();
        if(solve()) return true;
    }

    cout << "Backtracking from " << name[v] << " = (" << timeOf(value) << ", L" << labOf(value) << ")\n";
    domain=oldDomains;
    used[v]=0;
    current[v]=-1;
}
return false;
}

int main(){
    cout << "Initial domains:\n";
    printDomain();
    cout << "\n";
    solve();
    cout << "\nFinal schedule\n";
    cout << "Section Time Slot Lab\n";
    for(int i=0;i<3;i++) cout << name[i] << "          " << timeOf(current[i]) << "          L" << labOf(current[i]) << "
}

```

TSP_GA.cpp

```
#include <bits/stdc++.h>
using namespace std;

vector<double> x = {60,23,15,85,71,98,50,20,40,75,82,91,12,37,28,66,55,73,88,42};
vector<double> y = {200,45,150,90,123,45,220,30,180,155,60,120,210,100,77,89,130,140,33,170};

vector<vector<double>> d;
mt19937 gen(42);

vector<int> makeTour(){
    vector<int> a;
    for(int i=1;i<20;i++) a.push_back(i);
    shuffle(a.begin(),a.end(),gen);
    a.insert(a.begin(),0);
    return a;
}

double distTour(const vector<int>& a){
    double s=0;
    for(int i=0;i<20;i++) s += d[a[i]][a[(i+1)%20]];
    return s;
}

vector<int> ox(const vector<int>& a, const vector<int>& b){
    int n=a.size();
    vector<int> c(n,-1);
    uniform_int_distribution<int> pos(1,n-1);
    int l=pos(gen), r=pos(gen);
    if(l>r) swap(l,r);
    for(int i=l;i<=r;i++) c[i]=a[i];
    vector<int> rem;
    for(int i=1;i<n;i++){
        bool used=false;
        for(int j=l;j<=r;j++) if(c[j]==b[i]) used=true;
        if(!used) rem.push_back(b[i]);
    }
    int k=0;
    for(int i=r+1;i<n;i++) if(c[i]==-1) c[i]=rem[k++];
    for(int i=1;i<=r;i++) if(c[i]==-1) c[i]=rem[k++];
    c[0]=0;
    return c;
}

void mutate(vector<int>& a){
    uniform_real_distribution<double> q(0,1);
    if(q(gen)<0.15){
        uniform_int_distribution<int> pos(1,19);
        int i=pos(gen), j=pos(gen);
        swap(a[i],a[j]);
    }
}

int selectParent(const vector<vector<int>>& pop){
    vector<double> f(pop.size());
    double total=0;
    for(int i=0;i<(int)pop.size();i++){
        f[i]=1.0/distTour(pop[i]);
        total+=f[i];
    }
    uniform_real_distribution<double> q(0,total);
    double r=q(gen), s=0;
    for(int i=0;i<(int)pop.size();i++){
        s+=f[i];
        if(r<=s) return i;
    }
    return pop.size()-1;
}

pair<vector<int>,double> runGA(int popSize,int generations){
    vector<vector<int>> pop;
    for(int i=0;i<popSize;i++) pop.push_back(makeTour());
    for(int g=0;g<generations;g++){
        int best=0;
        for(int i=1;i<popSize;i++) if(distTour(pop[i])<distTour(pop[best])) best=i;
        vector<vector<int>> next;
        next.push_back(pop[best]);
        while((int)next.size()<popSize){
            int p1=selectParent(pop), p2=selectParent(pop);
            vector<int> child=ox(pop[p1],pop[p2]);
            mutate(child);
        }
    }
}
```

```

        next.push_back(child);
    }
    pop=next;
}
int best=0;
for(int i=1;i<popSize;i++) if(distTour(pop[i])<distTour(pop[best])) best=i;
return {pop[best],distTour(pop[best])};
}

int main(){
    d.assign(20,vector<double>(20));
    for(int i=0;i<20;i++) for(int j=0;j<20;j++)
        d[i][j]=sqrt((x[i]-x[j])*(x[i]-x[j])+(y[i]-y[j])*(y[i]-y[j]));

    vector<pair<int,int>> tests={{10,100},{20,100},{30,100},{10,200},{20,200},{30,200}};
    for(auto t:tests){
        auto ans=runGA(t.first,t.second);
        cout << "Population = " << t.first << ", Generations = " << t.second << "\n";
        cout << "Best tour: ";
        for(int v:ans.first) cout << v+1 << " ";
        cout << 1 << "\n";
        cout << fixed << setprecision(2) << "Total distance: " << ans.second << "\n\n";
    }
}

```